AXI-Lite FIR DSP Peripheral

Modular VHDL IP core implementing a register-controlled
FIR DSP interface with scalable architecture.

Design specification

Author: Gabriela Cabrera
Github: @MGabrielaCabrera
Date: March 2026
Version: 1.1

1. Project definition

This project focuses on the design and implementation of a synthesizable AXI-Lite slave peripheral in VHDL intended to control an external FIR DSP module through a structured register interface and dedicated control signals. The objective is to model a realistic system-on-chip integration scenario in which a processor communicates with a digital signal processing block via a standard bus protocol, while the DSP itself is implemented as a separate RTL component. In the first phase of the project, data and configuration transfers to the DSP are handled through internal signals driven by memory-mapped registers; in a second phase, the architecture will be extended to support high-throughput data paths using DMA or AXI-Stream interfaces.

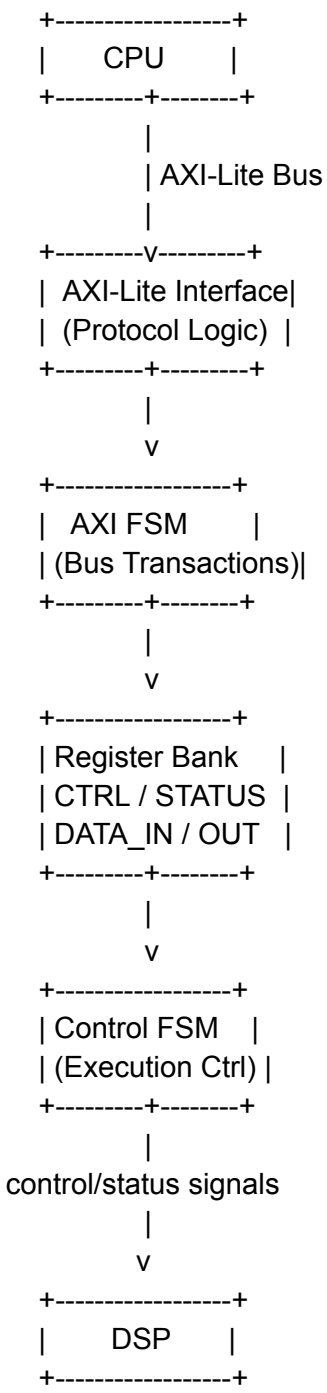
The peripheral exposes a set of memory-mapped registers accessible through AXI-Lite transactions, including control, status, coefficient, and configuration registers. A processor writes commands and parameters into these registers, and the peripheral's control logic interprets them to coordinate execution of the external FIR DSP. When enabled, the peripheral asserts control signals toward the DSP and monitors its status signals, such as ready or overflow, to supervise operation and ensure correct sequencing.

Internally, the design is partitioned into three main subsystems: an AXI protocol interface, a register bank, and a control finite state machine. The AXI interface handles address, data, and response channels using the VALID/READY handshake mechanism defined by the protocol. The register bank stores configuration values, coefficients, and control bits written by software, while the control FSM sequences interaction with the DSP, guaranteeing proper timing, synchronization, and safe coefficient updates. This separation of concerns improves readability, simplifies debugging, and allows each block to be verified independently.

The external FIR DSP module is connected to the peripheral through a dedicated signal interface that carries control, status, coefficient, and sample signals. Because the DSP resides outside the AXI peripheral module, it can be replaced, upgraded, or verified independently without modifying the bus interface. This modularity is essential in scalable hardware systems, where signal-processing cores may evolve or be swapped while maintaining a stable processor-facing control interface.

2. High-level overview

Block diagram



Module Architecture Description

TOP
├── axi_lite_slave_if
├── register_bank
├── control_fsm
├── dsp_if_adapter
└── dsp_pkg

Top-Level Module

The top-level module defines the external interface of the peripheral and structurally integrates all internal submodules. It instantiates the AXI-Lite slave interface, register bank, control finite state machine, DSP interface adapter, and shared package definitions, interconnecting them through clearly defined signals. This module contains no behavioral logic and serves strictly as a structural hierarchy layer to improve readability, maintainability, and system integration.

AXI-Lite Slave Interface

The AXI-Lite slave interface implements the full protocol required for communication with the processor, including independent read and write channels and VALID/READY handshake signaling. It decodes incoming transactions and converts them into simplified internal control signals such as read enable, write enable, address, and data buses. By isolating bus protocol handling in a dedicated block, the rest of the system remains independent of timing and signaling details specific to the AXI standard.

Register Bank

The register bank implements all memory-mapped registers visible to software, including control, status, coefficient, and configuration registers. It updates register values upon write transactions and returns stored values during read operations. The module also exposes individual register fields as structured outputs for internal logic, allowing other components to react directly to configuration changes. Its functionality is limited to storage and mapping, ensuring a clean separation from both protocol and execution logic.

Control Finite State Machine

The control FSM governs the operational sequencing of the external FIR DSP module. It interprets register contents, manages execution states, coordinates coefficient loading, and supervises DSP status signals such as ready or overflow. Based on these conditions, it generates deterministic control outputs including enable, reset, coefficient write strobes, and interrupt signals. This module encapsulates the behavioral logic of the peripheral and ensures safe, synchronized operation across all execution phases.

DSP Interface Adapter

The DSP interface adapter provides a dedicated interface layer between the internal control logic and the external DSP core. It formats signals, aligns handshakes, registers outputs, and performs any necessary timing adaptation to satisfy the DSP interface requirements.

This abstraction layer prevents tight coupling between system control logic and DSP implementation details, allowing the external processing module to be replaced or modified without impacting the rest of the design.

Shared Types and Constants Package

The shared package defines global constants, type declarations, register address maps, bit-field masks, and state enumerations used throughout the design. Centralizing these definitions ensures consistency across modules, reduces duplication, and simplifies maintenance. It also improves code clarity by replacing literal values with meaningful symbolic names, which is essential for scalable and reusable RTL development.

3. Requirements

Functional Requirements

FR-1 — The peripheral shall implement a fully compliant AXI-Lite slave interface supporting independent read and write channels.

FR-2 — The peripheral shall expose memory-mapped registers for control, status, configuration, and coefficient loading.

FR-3 — The peripheral shall allow software to enable and disable the DSP core.

FR-4 — The peripheral shall support runtime coefficient updates through register writes.

FR-5 — The peripheral shall provide real-time status feedback including ready, busy, and overflow indicators.

FR-6 — The peripheral shall prevent unsafe coefficient updates while DSP processing is active.

Interface Requirements

IR-1 — The module shall use a 32-bit AXI-Lite data bus.

IR-2 — The module shall support aligned 32-bit register accesses.

IR-3 — The DSP interface shall expose control, status, coefficient, and data signals.

IR-4 — All external signals shall be synchronous to the system clock.

Timing Requirements

TR-1 — All logic shall be synchronous to a single clock domain.

TR-2 — The design shall not include combinational paths between AXI inputs and outputs.

TR-3 — Register writes shall take effect within one clock cycle.

TR-4 — DSP control signals shall meet setup/hold requirements relative to the clock.

Reset Requirements

RR-1 — The design shall support an active-low synchronous reset.

RR-2 — After reset, all registers shall return to default values.

RR-3 — The DSP shall remain disabled after reset until explicitly enabled.

Safety and Robustness Requirements

SR-1 — Reserved register bits shall ignore writes and read as zero.

SR-2 — Invalid register accesses shall not cause undefined behavior.

SR-3 — Control signals shall never glitch during state transitions.

SR-4 — The system shall remain in a safe state if the DSP reports an error.

Scalability Requirements

SC-1 — The architecture shall allow replacement of the DSP module without modifying the AXI interface.

SC-2 — The design shall allow future addition of DMA or streaming data paths.

SC-3 — Register map shall reserve unused address space for future extensions.

4. Interface Specification

Inputs/outputs Ports

General

- **Clock and reset**
  clk : in std_logic
  rst_n : in std_logic

AXI-Lite Slave Interface

- **Write address**

  s_axi_awaddr : in std_logic_vector(31 downto 0)
  s_axi_awvalid : in std_logic
  s_axi_awready : out std_logic

- **Write data**

  s_axi_wdata : in std_logic_vector(31 downto 0)
  s_axi_wstrb : in std_logic_vector(3 downto 0)
  s_axi_wvalid : in std_logic
  s_axi_wready : out std_logic

- **Write response**

  s_axi_bresp : out std_logic_vector(1 downto 0)
  s_axi_bvalid : out std_logic
  s_axi_bready : in std_logic

- **Read address**

  s_axi_araddr : in std_logic_vector(31 downto 0)
  s_axi_arvalid : in std_logic
  s_axi_arready : out std_logic

- **Read data**

  s_axi_rdata : out std_logic_vector(31 downto 0)
  s_axi_rresp : out std_logic_vector(1 downto 0)
  s_axi_rvalid : out std_logic
  s_axi_rready : in std_logic

External DSP interface

- **Control**

  dsp_enable : out std_logic
  dsp_reset : out std_logic

dsp_mode : out std_logic_vector(1 downto 0)

- **Streaming data**

  dsp_data_in : out std_logic_vector(15 downto 0)
  dsp_data_in_valid : out std_logic
  dsp_data_in_ready : in std_logic

  dsp_data_out : in std_logic_vector(15 downto 0)
  dsp_data_out_valid : in std_logic
  dsp_data_out_ready : out std_logic

- **Coefficients**

  dsp_coeff_data : out std_logic_vector(15 downto 0)
  dsp_coeff_addr : out std_logic_vector(7 downto 0)
  dsp_coeff_we : out std_logic

Protocols

AXI-Lite Slave Protocol

The peripheral implements a fully compliant AXI4-Lite slave interface as defined by the AMBA AXI4 specification.

The interface supports independent read and write address/data channels using the VALID/READY handshake mechanism. All channels are synchronous to a single clock domain.

The following AXI-Lite features are supported:

- 32-bit data bus width
- 32-bit address width
- Single-beat transactions only
- Separate read and write channels
- Byte strobes (WSTRB) supported

The following features are not supported:

- Burst transactions
- Exclusive accesses
- Out-of-order responses
- Multiple outstanding transactions

All interface outputs are registered. No combinational paths exist between AXI inputs and outputs.

DSP Control Interface Protocol

The peripheral communicates with the external FIR DSP module using a simple synchronous control protocol.

The control protocol operates as follows:

1. Software writes configuration registers.
2. When the START condition is detected, the control FSM asserts dsp_start.
3. The DSP asserts dsp_busy while processing.
4. When computation completes, the DSP asserts dsp_done.
5. The FSM captures completion and updates status flags.

All signals are synchronous to the system clock. No asynchronous handshaking is used.

Coefficient updates are only permitted when dsp_busy = 0.

5. Functional Description

AXI-Lite Handshake

The interface supports independent read and write transactions using five separate channels:

- Write Address Channel (AW)
- Write Data Channel (W)
- Write Response Channel (B)
- Read Address Channel (AR)
- Read Data Channel (R)

All channels operate synchronously to a single clock domain and use the VALID/READY handshake mechanism. A transfer on any channel occurs on the rising edge of the clock when both VALID and READY are simultaneously asserted.

Write Address Channel (AW)

The master asserts:

- *AWVALID* to indicate a valid address
- *AWADDR* containing the target register address

The slave asserts:

- *AWREADY* when the Write FSM is in a state that can accept a new address (IDLE or W_WAIT)

The address transfer occurs when:

***AWVALID* = 1 and *AWREADY* = 1**

On the rising clock edge where the handshake occurs, the slave latches AWADDR into an internal register and captures the associated error flags (alignment check and address-range check). AWREADY is deasserted in the same cycle to prevent a second address from being accepted before the current transaction completes.

Write Data Channel (W)

The master asserts:

- *WVALID*
- *WDATA*
- *WSTRB*

The slave asserts:

- *WREADY* when the Write FSM is in a state that can accept write data (IDLE or AW_WAIT).

The data transfer occurs when:

***WVALID* = 1 and *WREADY* = 1**

The byte-strobe mask is applied combinationally before the data is registered: only byte lanes for which the corresponding WSTRB bit is '1' are written into the internal data register. Unselected lanes are forced to zero, ensuring full AXI4-Lite WSTRB compliance. WREADY is deasserted in the same cycle as acceptance.

Write Response Channel (B)

After both address (AW) and data (W) have been accepted, the Write FSM advances to the EXEC state, where the write operation is performed and a response is generated. The slave then asserts:

- *BVALID* to signal that the response is available.
- *BRESP* set to OKAY (2'b00) for a successful transaction, or SLVERR (2'b10) for an invalid or misaligned address

The master asserts:

- *BREADY* when it can accept the response

The response transfer occurs when:

***BVALID* = 1 and *BREADY* = 1**

BVALID is deasserted on the clock edge following a successful BREADY handshake. The Write FSM then returns to IDLE, ready for the next transaction.

Read Address Channel (AR)

The master asserts:

- *ARVALID* to indicate that a valid read address is present.
- *ARADDR* containing the target register address.

The slave asserts:

- *ARREADY* when the Read FSM is in IDLE state

The address transfer occurs when:

***ARVALID* = 1 and *ARREADY* = 1**

On the handshake clock edge, ARADDR is registered and the alignment and address-range checks are latched. ARREADY is deasserted immediately to prevent acceptance of a second read address while the current transaction is in progress.

Read Data Channel

After accepting the read address, the Read FSM drives the register read enable for one clock cycle (EXEC state) and then presents the result on the data channel. The slave asserts:

- *RVALID* to indicate that read data is available.
- *RDATA*
- *RRESP* set to OKAY (2'b00) or SLVERR (2'b10) depending on the address validity.

The master asserts:

- *RREADY*

The data transfer completes when:

***RVALID* = 1 and *RREADY* = 1**

RVALID, RDATA, and RRESP are held stable until the RREADY handshake occurs. RVALID is deasserted on the following clock edge and the Read FSM returns to IDLE.

Handshake Rules

The *VALID*/*READY* protocol follows these rules:

1. *VALID* must remain asserted until *READY* is asserted.
2. *READY* may be asserted independently of *VALID*.
3. A transfer occurs only when both *VALID* and *READY* are high.
4. All outputs are registered and synchronous.
5. No combinational dependency exists between interface inputs and outputs.

Write FSM

The Write FSM controls all three write channels (AW, W, B). It has eight states:

- **IDLE** — The FSM enters this state on reset or after a transaction completes. Both AWREADY and WREADY are asserted, indicating the slave is ready to accept either channel. The FSM waits until AWVALID, WVALID, or both are asserted by the master.
- **HANDSHAKE** — Both AWVALID and WVALID were detected simultaneously in IDLE. In this state both AWREADY and WREADY are asserted for one cycle, the address and strobed write data are latched, and the address error flag is evaluated. The FSM advances unconditionally to EXEC on the next clock edge.
- **AW_HANDSHAKE** — Only AWVALID was detected in IDLE. In this state AWREADY is asserted for one cycle, the write address is latched, and the address error flag is evaluated. WREADY remains deasserted. The FSM then advances to AW_WAIT to wait for the write data.

- **W_HANDSHAKE** — Only WVALID was detected in IDLE. In this state WREADY is asserted for one cycle and the strobed write data is latched. AWREADY remains deasserted. The FSM then advances to W_WAIT to wait for the write address.
- **AW_WAIT** — The AW handshake has completed and the write address has been latched, but write data has not yet arrived. AWREADY is deasserted to prevent acceptance of a second address. WREADY remains asserted so the slave can accept data as soon as the master presents it. The FSM exits when WVALID is asserted.
- **W_WAIT** — Write data has been accepted and latched, but the write address has not yet arrived. WREADY is deasserted to prevent acceptance of a second data beat. AWREADY is asserted so the address can be accepted as soon as the master presents it. The FSM exits when AWVALID is asserted.
- **EXEC** — Both address and data are available. If no error was detected, the register write enable (reg_write_en) is pulsed for exactly one clock cycle and reg_waddr and reg_wdata are driven to the register file. BRESP is set to OKAY or SLVERR according to the latched error flags. BVALID is asserted and the FSM advances unconditionally to BRESP on the next clock edge.
- **BRESP** — BVALID is held asserted while the FSM waits for the master to assert BREADY. Once the handshake occurs, BVALID is deasserted and the FSM returns to IDLE, ready for the next transaction.

Key implementation detail: AWREADY is only deasserted when an AW handshake is accepted, and WREADY is only deasserted when a W handshake is accepted. Neither signal is re-asserted until the FSM reaches a state where a new acceptance is valid, preventing double-acceptance of either channel.

In IDLE, if AWVALID and WVALID are asserted simultaneously, the FSM transitions to HANDSHAKE where both channels are accepted in a single cycle, then advances directly to EXEC, skipping AW_HANDSHAKE, W_HANDSHAKE, AW_WAIT and W_WAIT.

Read FSM

The Read FSM controls both read channels (AR, R). It has six states:

- **IDLE**. The initial and resting state after reset or at the end of a completed transaction. ARREADY is asserted, indicating the FSM is ready to accept a new read address. The FSM waits in this state until ARVALID is detected from the master.
- **HANDSHAKE**. ARVALID has been detected. In this state ARREADY is asserted for one cycle, the read address is latched into an internal register, and the address error flag is evaluated. The FSM advances unconditionally to EXEC on the next clock edge.
- **EXEC**. The read address has been latched and ARREADY deasserted. The FSM asserts reg_read_en for exactly one clock cycle and drives reg_raddr to the register file. RRESP is set according to the latched error flag. The FSM advances unconditionally to WAITING_DATA_STABLE to allow the register file output to propagate.

- **WAITING_DATA_STABLE**. A single pipeline bubble cycle inserted to allow the register file read data to settle after reg_read_en was asserted. No outputs change in this state. The FSM advances unconditionally to DATA_STABLE.
- **DATA_STABLE**. The register file output is now stable. RDATA is sampled and registered, RVALID is asserted, and the FSM advances unconditionally to RRESP.
- **RRESP**. RVALID, RDATA, and RRESP are held stable while the FSM waits for the master to assert RREADY. Once the handshake completes, RVALID is deasserted and the FSM returns to IDLE, ready to accept a new read transaction.

Error flags are latched at the moment the address handshake occurs (AW_HANDSHAKE or W_WAIT for write; HANDSHAKE for read). The BRESP/RRESP value is set when the FSM enters the EXEC state and remains stable until the response handshake completes.

When a write error is detected, BVALID is still asserted and the response handshake proceeds normally; the register write enable (reg_write_en) is NOT asserted, protecting the register file from an erroneous write.

Timing Behavior

- The module supports one outstanding write transaction and one outstanding read transaction simultaneously, as the two FSMs are fully independent.
- Back-pressure is supported on all channels through READY deassertion. The master may hold VALID asserted indefinitely while waiting for READY.
- The write register pulse (reg_write_en) is exactly one clock cycle wide, generated in the EXEC state of the Write FSM.
- The read enable pulse (reg_read_en) is exactly one clock cycle wide, generated in the EXEC state of the Read FSM.
- Address validation (alignment check, range decode) is performed combinationally on the captured address register and latched at handshake time, so the result is stable for the lifetime of the transaction.

DSP FSM

In the initial implementation phase, input samples are written by software through a memory-mapped DATA_IN register. Each write operation generates an internal strobe that feeds the DSP input interface using a ready/valid handshake mechanism. Output samples are captured into a DATA_OUT register for software retrieval. This approach simplifies integration while avoiding the complexity of streaming or DMA-based data movement.

The control logic shall be implemented as a three-process Mealy FSM with four states: IDLE, SEND, WAIT_RESULT and LOAD_COEFF. A Mealy architecture shall be used so that outputs respond to input conditions within the same clock cycle rather than waiting for the following one, reducing the number of cycles required to complete each handshake and improving the overall efficiency of the ready/valid protocol with the DSP. The three passthrough signals dsp_enable, dsp_reset and dsp_mode shall be driven as concurrent

assignments that permanently mirror their corresponding register map values, independently of the current FSM state and with no additional clock cycle of latency.

The STATUS register shall be updated combinationally from next_state and the current input strobes. STATUS.READY shall be asserted when the FSM is heading to IDLE and no strobe is active. STATUS.BUSY shall be asserted when the FSM is transitioning to SEND, WAIT_RESULT or LOAD_COEFF. STATUS.ERROR shall be asserted when an invalid operation is attempted from IDLE: a coefficient write while CTRL.ENABLE is asserted, or a sample write while CTRL.ENABLE is deasserted.

To send an input sample to the DSP filter, the AXI-Lite module shall generate a sample_write_strobe pulse when a new value is written to the DATA_IN register. At that moment, the sample shall be captured into an internal register so that it remains stable throughout the subsequent handshake, regardless of any later changes to DATA_IN. If sample_write_strobe is asserted and CTRL.ENABLE is set, the FSM shall transition from IDLE to SEND on the next clock edge. In the SEND state, dsp_data_in_valid shall be asserted and the captured sample shall be driven onto dsp_data_in. The FSM shall hold in SEND, keeping the data stable, until the DSP asserts dsp_data_in_ready to complete the handshake. Once the handshake is acknowledged, dsp_data_in_valid shall be deasserted and the FSM shall advance to WAIT_RESULT.

In the WAIT_RESULT state the FSM shall hold, keeping STATUS.BUSY asserted, until the DSP presents a valid output by asserting dsp_data_out_valid. The output capture logic shall detect the rising edge of dsp_data_out_valid so that the result is latched exactly once regardless of how many cycles the signal remains high. On that rising edge, the output sample shall be captured into the DATA_OUT register, dsp_data_out_ready shall be pulsed high for exactly one clock cycle to acknowledge the transfer, and reg_data_out_strobe shall be simultaneously asserted for one cycle to notify the register interface that new data is available. The FSM shall then return unconditionally to IDLE.

Coefficient loading shall follow a separate path. The AXI-Lite module shall generate a coeff_write_strobe pulse when a new value is written to the COEFF_DATA register. If at that moment CTRL.ENABLE is deasserted — meaning the DSP is disabled — the FSM shall transition from IDLE to LOAD_COEFF. In that state, the value of COEFF_DATA shall be driven onto dsp_coeff_data, the value of COEFF_ADDR onto dsp_coeff_addr, and dsp_coeff_we shall be pulsed high for exactly one clock cycle. The FSM shall then return to IDLE unconditionally on the following clock edge. Coefficient writes shall be blocked when CTRL.ENABLE is asserted, preventing filter reconfiguration while the DSP is active. Attempting a coefficient write in that condition shall leave the FSM in IDLE and assert STATUS.ERROR.

This method is inherently slow, fully CPU-dependent, and does not scale for high-throughput applications. Since each sample transfer requires explicit software intervention through memory-mapped register writes and reads, overall performance is limited by processor bandwidth and latency. However, despite these limitations, this approach is well suited for phase 1 of this project, where simplicity, observability, and ease of debugging are prioritised over performance. It enables straightforward validation of the control logic, DSP integration,

and register interface before introducing more complex data movement mechanisms such as DMA or AXI-Stream in later development stages.

6. Register Map

Address	Name	Bits	Access	Reset	Description
0x00	CTRL	[0]	RW	0	Enable DSP
		[1]	RW	0	Reset DSP
		[3:2]	RW	0	Mode select 00: Normal FIR filtering 01: Bypass 10: Test pattern 11: Reserved
		[31:4]	—	0	Reserved
0x04	STATUS	[0]	RO	0	Ready
		[1]	RO	0	Error
		[2]	RO	0	Busy
		[31:3]	—	0	Reserved
0x08	COEFF_DATA	[15:0]	RW	0	Coefficient value
		[31:16]	—	0	Reserved
0x0C	COEFF_ADDR	[7:0]	RW	0	Coefficient index
		[31:8]	—	0	Reserved
0x10	DATA_IN	[15:0]	RW	0	DSP Data in
0x14	DATA_OUT	[15:0]	RO	0	DSP Data out

7. Reset and Initialization Behavior

The design implements a synchronous, active-low reset signal (rst_n). All internal state elements are reset on the rising edge of the system clock when rst_n is asserted low. The reset is assumed to be stable and synchronous to the clock domain.

Register Reset Values

After a reset:

- The DSP is disabled: CTRL.ENABLE = ‘0’
- All configuration registers are cleared to default values.
- The peripheral reports READY = ‘1’.
- Internal flags signals are cleared: sample_write_strobe and coef_write_strobe.
- No pending AXI transactions remain active.

AXI Interface Behavior During Reset

During reset assertion:

- All AXI output signals are driven to zero.
- The slave does not acknowledge transactions.
- Any ongoing transaction is aborted and must be reissued by the master after reset deassertion.

Reset Release Sequencing

After reset deassertion:

- The control FSM transitions to the IDLE state.
- The peripheral remains inactive until explicitly enabled by the CPU.
- No automatic coefficient loading is performed.
- The CTRL.ENABLE keeps inactive until the CPU changes it.

Verification Strategy

a. testbench approach
b. coverage plan
c. assertions
d. corner cases

Future Extensions

e. DMA support
f. streaming mode
g. interrupts